

BigQuery ML (BQML)

A Simple, Three-Model Walkthrough — Linear Regression, Logistic Regression, K-Means

This guide trains three simple models directly from the BigQuery web console SQL editor, using the same synthetic `partition_demo.orders_raw` table from the partitioning/clustering demo (`order_id`, `order_date`, `order_ts`, `customer_id`, `country`, `category`, `amount`). No Python, no notebook, no separate ML tooling — BQML models are created, evaluated, and queried with plain SQL.

Honest caveat to set with the class up front: `amount`, `country`, `category`, and `date` were all generated with independent random draws, so there is no real signal linking them. Expect low R^2 and close-to-chance classification accuracy. That's fine — the goal here is the *procedure* (train → evaluate → predict → inspect), not a meaningful business result. On real data the same six commands per model are what you'd run.

The BQML workflow, in general

Every BQML model, regardless of type, follows the same shape:

1. **CREATE MODEL** — one SQL statement; the `SELECT` is the training data, one column is the label (for supervised models), the rest are features. Training runs as a background job.
2. **ML.EVALUATE** — run held-out or training data back through the model to get quality metrics.
3. **ML.PREDICT** — score new rows.
4. **Inspect** — `ML.TRAINING_INFO` (loss curve per iteration), `ML.WEIGHTS` (feature coefficients), `ML.FEATURE_INFO` (per-feature stats BQML computed automatically).

Linear Regression — Predict Order Amount

Business framing: "Can we predict how much an order will be worth, from when/where/what category it's in?"

Why this model type

The label we're predicting, amount, is a continuous numeric value — not a category. Whenever the target is a number on a continuous scale (price, revenue, duration, temperature), regression is the right family, and linear regression is the natural first model to reach for: it's the fastest to train, the easiest to explain (a weighted sum of the features), and gives you a baseline to beat before trying anything fancier like boosted trees (model_type='boosted_tree_regressor') or a DNN.

Step 1 — Train

```
CREATE OR REPLACE MODEL `partition_demo.amount_predictor`  
OPTIONS(  
  model_type = 'linear_reg',  
  input_label_cols = ['amount'],  
  data_split_method = 'AUTO_SPLIT' -- BQML auto-reserves data for evaluation  
) AS  
SELECT  
  country,  
  category,  
  EXTRACT(MONTH FROM order_date) AS order_month,  
  EXTRACT(DAYOFWEEK FROM order_date) AS order_dow,  
  amount  
FROM `partition_demo.orders_raw`;
```

BQML automatically one-hot encodes the string columns (country, category), infers this is a regression problem because the label is numeric, and splits data for training vs. evaluation. No manual encoding or train/test split needed.

Step 2 — Evaluate

```
SELECT * FROM ML.EVALUATE(MODEL `partition_demo.amount_predictor`);
```

Yardsticks this returns, and what each one means:

Metric	What it tells you
mean_absolute_error	Average absolute difference between predicted and actual amount, in the same units as amount (rupees/dollars). Easiest one to explain to a business audience.
mean_squared_error	Like MAE but squares each error first, so large misses are penalized much more heavily than small ones. Same units-squared, harder to interpret directly.
mean_squared_log_error	MSE computed on log-transformed values — useful when the label spans orders of magnitude and you care about relative, not absolute, error.

median_absolute_error	Like MAE but the median instead of the mean — much less sensitive to a few extreme outlier orders skewing the picture.
r2_score	Fraction of the variance in amount the model explains, from roughly 0 (no better than guessing the average) to 1 (perfect fit). Can go negative if the model is worse than the average. Expect this near zero on our random data — that's the correct, honest result.
explained_variance	Similar to R^2 , focuses purely on variance explained without penalizing bias in the same way; usually read alongside r2_score rather than instead of it.

Step 3 — Predict

```
SELECT
  country, category, order_month, order_dow,
  predicted_amount
FROM ML.PREDICT(MODEL `partition_demo.amount_predictor`,
  (SELECT country, category,
    EXTRACT(MONTH FROM order_date) AS order_month,
    EXTRACT(DAYOFWEEK FROM order_date) AS order_dow
  FROM `partition_demo.orders_raw`
  LIMIT 10));
```

Step 4 — Inspect the model internals

```
-- Loss per training iteration (should trend down and plateau)
SELECT * FROM ML.TRAINING_INFO(MODEL `partition_demo.amount_predictor`);

-- Learned coefficient (weight) per feature
SELECT * FROM ML.WEIGHTS(MODEL `partition_demo.amount_predictor`);

-- Auto-computed stats BQML used per feature (min/max/mean/category counts)
SELECT * FROM ML.FEATURE_INFO(MODEL `partition_demo.amount_predictor`);
```

ML.TRAINING_INFO is a good live-demo moment: if the loss curve is flat from iteration 1, that's visible, immediate evidence to the class that the features carry little signal — which matches the r2_score near zero and reinforces why the metric exists.

Logistic Regression — Classify High-Value Orders

Business framing: "Is this order likely to be a high-value order (say, over 2,500), yes or no?" — a binary classification problem instead of a continuous prediction.

Why this model type

We're deliberately reframing the same underlying data as a classification problem to contrast it with Model 1: the label here, `is_high_value`, is 0/1, not a continuous number, so linear regression is the wrong tool — logistic regression is the direct binary-outcome counterpart, modeling the *probability* of the positive class rather than a raw number. It's the standard first choice for any yes/no outcome (churn, fraud, click/no-click, high-value/not) for the same reasons linear regression is the first choice for continuous ones: fast, interpretable, a solid baseline before `boosted_tree_classifier` or a DNN.

Step 1 — Train

```
CREATE OR REPLACE MODEL `partition_demo.high_value_classifier`
OPTIONS(
  model_type = 'logistic_reg',
  input_label_cols = ['is_high_value'],
  data_split_method = 'AUTO_SPLIT'
) AS
SELECT
  country,
  category,
  EXTRACT(MONTH FROM order_date) AS order_month,
  IF(amount > 2500, 1, 0) AS is_high_value
FROM `partition_demo.orders_raw`;
```

BQML detects the 0/1 label and switches automatically to classification — same CREATE MODEL shape as Model 1, only `model_type` and the label's nature changed.

Step 2 — Evaluate

```
SELECT * FROM ML.EVALUATE(MODEL `partition_demo.high_value_classifier`);
```

Metric	What it tells you
precision	Of the orders the model flagged as high-value, what fraction actually were. High precision = few false alarms.
recall	Of the orders that actually were high-value, what fraction the model caught. High recall = few missed cases.
accuracy	Overall fraction of predictions (both classes) that were correct. Can be misleading if the classes are imbalanced — worth pairing with precision/recall, not using alone.
f1_score	Harmonic mean of precision and recall — a single number when you need to balance both rather than reading two separate ones.

log_loss	Penalizes confident-but-wrong predictions heavily, rewards confident-and-right ones. Lower is better; sensitive to how well-calibrated the predicted probabilities are, not just the final yes/no call.
roc_auc	Probability the model ranks a random positive example above a random negative one, across every possible decision threshold. 0.5 = coin flip, 1.0 = perfect separation. Expect close to 0.5 on our random data.

Threshold matters: ML.EVALUATE defaults to a 0.5 cutoff for turning a probability into a yes/no. You can re-run it at a different cutoff to see the precision/recall trade-off shift:

```
SELECT * FROM ML.EVALUATE(MODEL `partition_demo.high_value_classifier`,
  TABLE `partition_demo.orders_raw`,
  STRUCT(0.3 AS threshold));
```

Step 3 — Confusion matrix and ROC curve

```
-- Full breakdown of true/false positives/negatives
SELECT * FROM ML.CONFUSION_MATRIX(MODEL `partition_demo.high_value_classifier`);

-- Precision/recall at every threshold – the data behind the ROC curve
SELECT * FROM ML.ROC_CURVE(MODEL `partition_demo.high_value_classifier`);
```

The confusion matrix is the most intuitive artifact to show a class new to classification metrics: it's just four counts (true positive, false positive, true negative, false negative), and every metric above is arithmetic derived from those four numbers.

Step 4 — Predict

```
SELECT
  country, category, order_month,
  predicted_is_high_value,
  predicted_is_high_value_probs
FROM ML.PREDICT(MODEL `partition_demo.high_value_classifier`,
  (SELECT country, category,
    EXTRACT(MONTH FROM order_date) AS order_month
  FROM `partition_demo.orders_raw`
  LIMIT 10));
```

predicted_is_high_value_probs is worth pointing out specifically — BQML returns the full probability for each class, not just the final 0/1 call, which is what ML.EVALUATE's threshold option is operating on underneath.

K-Means Clustering — Customer Segmentation

Business framing: "Are there natural groups of customers based on how often they order and how much they spend, without us defining the groups in advance?"

Why this model type

This is a fundamentally different problem from Models 1 and 2: there's no label to predict, no known right answer per row. We just want the data to reveal its own structure — customers who behave similarly, grouped together. That's unsupervised learning, and k-means is the standard, simplest algorithm for it: pick a number of groups (k), and it iteratively assigns each row to whichever group's centroid is closest, then recomputes centroids, until it stabilizes.

Worth flagging explicitly to the class: this is a completely different sense of the word "clustering" from the storage *clustering* in the partitioning/clustering guide. BQML clustering groups similar rows together conceptually; storage clustering physically sorts rows on disk for pruning. Same word, unrelated concepts — a common point of confusion worth calling out directly.

Step 1 — Build customer-level features

K-means needs one row per entity you're segmenting (customers), not one row per order, so we aggregate first.

```
CREATE OR REPLACE TABLE `partition_demo.customer_features` AS
SELECT
  customer_id,
  COUNT(*)      AS order_count,
  AVG(amount)   AS avg_amount,
  COUNT(DISTINCT category) AS category_variety
FROM `partition_demo.orders_raw`
GROUP BY customer_id;
```

Step 2 — Train

```
CREATE OR REPLACE MODEL `partition_demo.customer_segments`
OPTIONS(
  model_type = 'kmeans',
  num_clusters = 4,
  standardize_features = TRUE  -- puts order_count/avg_amount on comparable scales
) AS
SELECT order_count, avg_amount, category_variety
FROM `partition_demo.customer_features`;
```

`standardize_features` matters more here than in Models 1-2: k-means measures literal distance between points, so a feature ranging in the thousands (`avg_amount`) would dominate one ranging in single digits (`order_count`) unless both are scaled first.

Step 3 — Evaluate (no labels, so different yardsticks)

```
SELECT * FROM ML.EVALUATE(MODEL `partition_demo.customer_segments`);
```

There's no "correct answer" to check against, so evaluation here measures how tight and well-separated the clusters are, not prediction accuracy:

Metric	What it tells you
davies_bouldin_index	Average similarity between each cluster and its most-similar other cluster. Lower is better — it means clusters are compact and well-separated from each other, rather than blurring together.
mean_squared_distance	Average squared distance from each point to its assigned cluster's centroid — how tightly packed the clusters are internally. Lower means tighter clusters, but trivially shrinks toward zero as k grows, so it's read alongside k, not alone.

Choosing k: the elbow method

Because mean_squared_distance always improves as k increases, picking k isn't about minimizing it outright — it's about finding the point where adding another cluster stops helping much. Train the same model at a few different k values and compare:

```
CREATE OR REPLACE MODEL `partition_demo.customer_segments_k3`
OPTIONS(model_type='kmeans', num_clusters=3, standardize_features=TRUE) AS
SELECT order_count, avg_amount, category_variety FROM `partition_demo.customer_features`;

CREATE OR REPLACE MODEL `partition_demo.customer_segments_k5`
OPTIONS(model_type='kmeans', num_clusters=5, standardize_features=TRUE) AS
SELECT order_count, avg_amount, category_variety FROM `partition_demo.customer_features`;

-- Compare mean_squared_distance across k=3, k=4, k=5 from ML.EVALUATE on each
-- The "elbow" -- where the curve stops dropping sharply -- is the natural choice for k.
```

Step 4 — Predict (assign each customer to a segment)

```
SELECT customer_id, order_count, avg_amount, category_variety, CENTROID_ID AS segment
FROM ML.PREDICT(MODEL `partition_demo.customer_segments`,
  TABLE `partition_demo.customer_features`)
LIMIT 20;
```

Step 5 — Inspect the segments themselves

```
-- The actual centroid (profile) of each segment -- this is how you'd
-- describe each segment in plain language, e.g. "Segment 2: frequent,
-- low-value, single-category shoppers"
SELECT * FROM ML.CENTROIDS(MODEL `partition_demo.customer_segments`);
```

ML.CENTROIDS is the payoff moment for this demo: each row is one segment's average order_count, avg_amount, and category_variety, which is exactly the language you'd use to name and act on the segments in a real business review.

Recap: Same Workflow, Three Problem Types

Aspect	Linear Regression	Logistic Regression	K-Means
Learning type	Supervised	Supervised	Unsupervised
Label	Continuous (amount)	Binary (0/1)	None
Question answered	How much?	Yes or no?	What natural groups exist?
model_type	linear_reg	logistic_reg	kmeans
Key eval metrics	r2_score, MAE, MSE	precision, recall, roc_auc	davies_bouldin_index
Predict output	A number	Class + probability	A cluster ID

Every model, regardless of row above, was created with one CREATE MODEL statement, evaluated with ML.EVALUATE, scored with ML.PREDICT, and inspected with a type-specific function (ML.WEIGHTS / ML.CONFUSION_MATRIX / ML.CENTROIDS). That consistency is the main thing worth landing with the class — the SQL surface doesn't change much across very different problem types; what changes is model_type, what the label looks like, and which metrics matter for judging the result.

Housekeeping: listing and removing models

```
-- List all models in the dataset
SELECT * FROM `partition_demo.INFORMATION_SCHEMA.MODELS`;

-- Drop a model when done
DROP MODEL `partition_demo.amount_predictor`;
```